

User Profile & Onboarding System

Read-only investigation of how user roles, signup, onboarding, and downstream profile state are wired together. No source files were modified.

Audit type: Read-only investigation **Scope:** app/, lib/, components/ **Files modified:** 0

Contents

- 1. Fields collected per user type
- 2. Onboarding flow map
- 3. Signup flow
- 4. How user type is determined
- 5. Type definitions
- 6. Summary of issues

STEP 1

Fields collected per user type

 Client user fields

At signup • app/signup/page.tsx

Field	Required	Source
firstName	required	text input
lastName	required	text input
email	required	email input
password	required (≥8 chars)	password input
confirmPassword	required, must match	password input
role	—	Toggle (default 'client')
isOnboarded	—	set false at creation
ownsShop	—	set false at creation

At onboarding • app/onboarding/client/page.tsx

Field	Required	Stored as
firstName / lastName	display only	read-only, carried from signup
phone	required	"+{code} {number}"
phoneCountryCode	required	string
country	required	ISO code
city	required	string
languages	optional	defaults to ["English"]
isOnboarded	—	set true on submit

Barber user fields

At signup: same fields as client; role toggle set to 'barber'.

Onboarding step 1 — Basic info • app/onboarding/barber/page.tsx

Field	Required (UI)	Notes
firstName / lastName	read-only	carried from signup
phone + phoneCountryCode	NOT validated	falls back to null
country	NOT validated	falls back to "Unknown"
city	NOT validated	falls back to "Unknown"
bio	NOT validated	falls back to "Professional barber."
languages	optional	defaults to ["English"]

Step 2 — Vibe & specialties

Field	Required	Type
vibes (e.g. 🤘 Cool & chill)	optional	string[]
specialties (e.g. Skin Fade)	optional	string[]
clientele (e.g. 🧒 Kids 3–12)	optional	string[]

Step 3 — Services

Field	Required	Notes
<code>titeZMeCut.durationMinutes</code> + <code>price</code>	required	defaults 45 min / €20
Custom services (name / duration / price)	optional	2 sample services pre-filled; written to <code>services</code>

Step 4 — Launch

Creates `barberProfiles/{uid}`, schedule shard, services, then sets `isOnboarded: true`, `isLive: false`, `approvalStatus: 'pending'`, and generates a unique `barberCode: TZB-XXXXXX`.

Shop owner — not a separate role

Shop owner is **not a user type**. It's a derived flag `ownsShop: true` on a barber user. There is **no dedicated onboarding flow**. The shop is created later via `app/dashboard/barber/create-shop/page.tsx`.

Field	Required	Notes
<code>name</code> (shop name)	required	sanitized, max 100
<code>contactPhone</code> + <code>phoneCountryCode</code>	required	—
<code>address.street</code>	required	—
<code>address.number</code> (building)	required	—
<code>address.postalCode</code>	required	—
<code>address.floor</code>	optional	—
<code>address.city</code> / <code>country</code>	required	—
<code>googleMapsUrl</code>	auto	auto-generated from address
<code>description</code>	optional	max 2000 chars
<code>coverPhotoUrl</code> / <code>photos</code>	optional	empty array at creation

Creating a shop sets `users/{uid}.ownsShop = true` and `barberProfiles/{uid}.ownsShop = true`, `shopId = uid`.

STEP 2 Onboarding flow map

```
app/onboarding/  
├─ barber/page.tsx  — 4-step wizard, BARBER role  
└─ client/page.tsx  — single-page form, CLIENT role
```

No shop-owner onboarding exists. Shop ownership is added post-onboarding.

Post-completion redirects

Page	After submit
<code>onboarding/client/page.tsx</code>	<code>window.location.href = '/dashboard/client'</code> (line 132 — hard reload to refresh AuthContext)
<code>onboarding/barber/page.tsx</code>	<code>window.location.href = '/dashboard/barber'</code> (line 228)

Redirect-to-onboarding logic • `components/RouteGuard.tsx:93-106`

If `appUser.isOnboarded === false` AND route is not `/onboarding/*`, `/`, `/login`, `/signup`:

- `role === 'client'` → redirect to `/onboarding/client`
- `role === 'barber'` → redirect to `/onboarding/barber`

"Healing" mechanism • `lib/auth-context.tsx:102-115`

If user has `role='barber'` AND `isOnboarded=false` BUT a `barberProfiles/{uid}` doc exists → auto-flips `isOnboarded` to `true`. Recovery path for orphaned half-completed onboardings.

Also: `app/api/migrate-barbers-onboarded/route.ts` is a backfill endpoint that sets `isOnboarded: true` on all `barberProfiles` with `isLive: true` that are missing the flag.

STEP 3 Signup flow

File: `app/signup/page.tsx` • Two signup paths.

Path A — Email + password

`handleSignup`, lines 147–211

1. Collects: `role` (toggle, default `client`), `firstName`, `lastName`, `email`, `password`, `confirmPassword`.
2. Calls Firebase `createUserWithEmailAndPassword`.
3. Writes `users/{uid}` with `{ uid, email, role, firstName, lastName, createdAt, isOnboarded: false, ownsShop: false }`.
4. Sends verification email via `sendEmailVerification`.
5. Shows verification screen — user must click email link, then log in.
6. **Does NOT redirect to onboarding directly.** Onboarding happens after the user later logs in (RouteGuard catches `isOnboarded: false`).

Path B — Google sign-in

`handleGoogleSignIn`, lines 110–145

1. Pops up Google auth, gets user.
2. If user doc does NOT exist: creates `users/{uid}` with `role: 'client'`, `isOnboarded: true` (skips onboarding), `profilePhotoUrl` from Google.
3. If user doc exists: reads role.

4. Redirects by role: `admin` → `/admin` , `barber` → `/dashboard/barber` , `client` → `/dashboard/client` .

Inconsistency: Google sign-in bypasses onboarding by setting `isOnboarded: true` immediately. Email signup leaves it `false` and forces them through onboarding. Google clients never get prompted for phone, city, country, or languages.

STEP 4 How user type is determined

The role system

Source of truth: `users/{uid}.role` — string enum.

Valid runtime values used in code: `'client'` , `'barber'` , `'admin'` .

Inconsistencies across the role definition:

- `firestore.rules:48` accepts `['client', 'barber', 'shop_owner', 'admin']` in `isValidUser()` — but `'shop_owner'` is never written by the app. CLAUDE.md confirms it should not exist.
- `lib/schemas.ts:6` : Zod enum allows only `['client', 'barber']` — no `admin` , no `shop_owner` .
- `lib/types.ts:36` : TypeScript type allows `'client' | 'barber' | 'admin'` .
- `lib/auth-context.tsx:16` : `AppUser.role` only allows `'client' | 'barber'` .
- `app/api/cleanup/route.ts:12` : queries `where('role', '==', 'shop_owner')` — dead/broken code.

How "shop owner" is determined

Not a role. It's a boolean flag `users/{uid}.ownsShop = true` AND `barberProfiles/{uid}.ownsShop = true` . The user is always still `role: 'barber'` .

Conditional UI based on `ownsShop`

File	Use
<code>app/dashboard/barber/layout.tsx:178</code>	<code>appUser?.ownsShop ?</code> — show shop nav
<code>components/BarberInvitesTab.tsx:125</code>	hide invites if already owns shop
<code>components/DeleteAccountButton.tsx:141</code>	different deletion path if owner
<code>app/dashboard/barber/create-shop/page.tsx:106</code>	redirects to <code>/dashboard/shop</code> if already owner
<code>app/api/delete-account/route.ts:21</code>	server-side check

Set to true by

- `app/dashboard/barber/create-shop/page.tsx:145, 150` — only place this happens for users.

Reset to false by

- `components/ShopTeamTab.tsx:214` (barber leaving shop)
- `components/ShopSettingsTab.tsx:316-317` (owner closing shop)

Admin role

Admins are determined via `role === 'admin'` OR `isAdmin === true` (see `firestore.rules:21-28` and `components/AdminGuard.tsx`). Admins log in via `/admin/login` only — `app/login/page.tsx:97` actively blocks admin accounts from the public login.

STEP 5 Type definitions

UserDocument • `lib/types.ts:33-53`

```
export interface UserDocument {
  uid: string;
  email: string;
  role: 'client' | 'barber' | 'admin';
  firstName: string;
  lastName: string;
  city: string;
  country: string;
  photoUrl?: string;
  phone?: string;
  phoneCountryCode?: string;
  isOnboarded: boolean;
  ownsShop: boolean;
  shopId?: string | null;
  favoriteBarbers: string[];
  noShowCount?: number;
  createdAt: number;
  notifyInviteResponse?: boolean;
  notifyNewBooking?: boolean;
  unreadCount?: number;
}
```

Note: `city`, `country`, and `favoriteBarbers` are typed as required but in practice are populated only after onboarding (or never, for Google-signup clients). The runtime `AppUser` interface in `auth-context.tsx` more loosely makes these optional.

BarberProfile • lib/types.ts:3-31

```
export interface BarberProfile {
  userId: string;
  bio: string;
  city: string;
  isLive: boolean;
  isSolo: boolean;
  ownsShop: boolean;
  shopId: string | null;
  languages: string[];
  vibe: string[];          // ⚠ singular "vibe" here
  specialties: string[];
  clientele: string[];
  photos: string[];
  rating: number;
  reviewCount: number;
  totalCuts: number;
  barberCode: string;
  currency: string;
  profilePhotoUrl?: string;
  instagram?: string;
  tiktok?: string;
  facebook?: string;
  showPhone?: boolean;
  titeZMeCut?: {
    durationMinutes: number;
    price: number;
    currency?: string;
  };
}
```

Type drift: BarberProfile.vibe is singular, but onboarding/barber/page.tsx:168 writes vibes (plural) and firestore.rules:63 reads vibes. Live code uses the plural form; the type is wrong/stale.

AppUser • lib/auth-context.tsx:13-43

Different from `UserDocument` — superset that merges user + barber profile fields (city, bio, isLive, languages, etc.) for convenience in components. Has `[key: string]: any` escape hatch.

Summary of issues

No fixes applied — just noted for follow-up.

#	Issue	Severity
1	'shop_owner' role accepted in firestore.rules:48 but never written. CLAUDE.md says it shouldn't exist.	Cleanup
2	app/api/cleanup/route.ts:12 queries where('role', '==', 'shop_owner') — guaranteed empty result, likely dead code.	Dead code
3	Google signup bypasses onboarding entirely. Clients land in dashboard with no phone/city/country/languages.	UX bug
4	Barber onboarding Step 1 has zero validation — defaults to "Unknown" city/country and "Professional barber." bio if user skips.	UX bug
5	BarberProfile.vibe (singular) in lib/types.ts:12 doesn't match vibes (plural) used everywhere else.	Type drift
6	app/onboarding/barber/page.tsx:137-140 calls setDoc with merge:true to set role: 'barber' during launch — but role is already set at signup. Defensive but redundant.	Minor
7	lib/schemas.ts:6 Zod role enum lacks 'admin', breaking admin role validation if userSchema.parse() is ever called on an admin user.	Type drift
8	Two separate Barber types: lib/types.ts BarberProfile (strict) vs lib/schemas.ts barberSchema (all optional, .passthrough()). The schema provides almost no validation.	Type drift
9	onboarding/client/page.tsx writes phone/country/city/languages/isOnboarded — but never writes favoriteBarbers: [] even though UserDocument types it as required.	Minor type drift

No source files were modified during this audit. Report generated directly from the working tree at the time of investigation.